

Uma API para hábitos e rotinas.

Referência prática dos endpoints HTTP de autenticação e do API Gateway, com exemplos de uso, autenticação, autorização por propriedade e integração dos serviços.

Do cliente aos microsserviços

HTTP/REST na entrada · Go e chi no Gateway · gRPC com TLS entre serviços · Node.js no serviço de autenticação.

Domínios documentados

Autenticação, usuários, social, hábitos, rotinas e estatísticas.

Stack principal

Go · Node.js · gRPC · Protocol Buffers · PostgreSQL · Redis · Docker.

Como a API está organizada

O cliente usa HTTP. O Gateway autentica as chamadas e encaminha operações de domínio aos serviços internos por gRPC. O serviço Auth é uma API Node.js separada que emite access e refresh tokens.

COMPONENTE	TECNOLOGIA / FUNÇÃO	PORTA LOCAL
Gateway	Go + chi · HTTP/REST	8084
Auth Service	Node.js + Express · tokens e autenticação	3000 interno
User Service	gRPC sobre TLS	8080 interno
Social Service	gRPC sobre TLS	8081 interno
Habit / Routine Service	gRPC sobre TLS	8082 interno
Stats Service	gRPC sobre TLS	8083 interno
Nginx	Proxy reverso para Auth e Gateway	80

Gateway direto

`http://localhost:8084`

Use para chamar endpoints do Gateway durante o desenvolvimento local.

Via Nginx

`http://localhost`

O proxy encaminha rotas Auth e Gateway pela porta 80.

Fluxo de autenticação

1. Crie uma conta ou faça login no Auth Service.
2. Nas rotas protegidas do Gateway, envie o header `Authorization` com o esquema `Bearer` seguido pelo `accessToken`.
3. Quando o access token expirar, envie o `refreshToken` para renovar o access token.

Pré-requisito de execução: o Gateway valida o token usando `JWT_SECRET`. Configure no container do Gateway o mesmo segredo usado pelo Auth Service; não inclua o valor real do segredo em documentação, exemplos ou repositórios.

Formato geral das respostas

```
{
  "timestamp": "2026-10-09T15:04:05Z",
  "api_version": "v1",
  "duration_ms": 12,
  "message": "habit created",
  "status": "success",
  "data": { },
  "http": { "method": "POST", "url": "/api/v1/habit/" }
}
```

Em erro, a resposta contém `status: "error"`, `message` e normalmente `error`. Alguns erros do Auth podem ter formato diferente.

Auth Service

Rotas do Auth passam pelo Nginx em `http://localhost`. Os caminhos públicos completos preservam o prefixo `/auth`.

MÉTODO	ROTA	BODY / OBSERVAÇÃO
POST	/api/v1/auth/signup	<code>{"name": "Ana", "email": "ana@example.com", "password": "senha-forte", "tel": "..."}"</code> name, email e password obrigatórios; tel opcional.
POST	/api/v1/auth/login	<code>{"email": "ana@example.com", "password": "senha-forte"}</code>
POST	/api/v1/auth/refresh	<code>{"refreshToken": "<refresh-token>"}</code>
POST	/api/v1/auth/logout	Header <code>Authorization</code> com access token; revoga o refresh token do usuário.
GET	/api/v1/auth/me	Header <code>Authorization</code> com access token; retorna o ID do usuário autenticado.

Exemplo de login

```
curl -X POST http://localhost/api/v1/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email": "ana@example.com", "password": "senha-forte"}'
```

A resposta de login/cadastro inclui `accessToken` e `refreshToken`. O access token expira em 30 minutos; o refresh token em 7 dias.

Porta interna: Auth escuta na porta 3000, mas o Compose de produção não a publica diretamente no host. Use a URL via Nginx acima.

Rotas auxiliares do Auth

MÉTODO	ROTA	OBSERVAÇÃO
GET	/helpers/lifeCheck	Health check (rota interna do Auth).
GET	/users/me	Dados do usuário autenticado; exige token (rota interna do Auth).
GET	/users/data/{id}	Busca usuário por ID (rota interna do Auth).

As rotas auxiliares são montadas no Auth Service, mas não são encaminhadas por uma location do Nginx na configuração atual; a porta 3000 também não é publicada pelo Compose de produção.

Auth usa JWT e Redis para refresh tokens. Nunca compartilhe tokens reais em collections ou documentação pública.

Usuários e social

Base do Gateway: `http://localhost:8084`. Todas as rotas desta seção exigem o header `Authorization` com esquema `Bearer` e o access token emitido pelo Auth.

Usuários · prefixo `/api/v1/user`

MÉTODO	ROTA	PARÂMETRO / BODY
GET	<code>/api/v1/user/GetUserByID/{id}</code>	ID do usuário na URL.
POST	<code>/api/v1/user/GetUsersByIDs</code>	<code>{"ids":["ID_1","ID_2"]}</code>
POST	<code>/api/v1/user/SearchUsers</code>	<code>{"name":"Ana","email":"ana@example.com"}</code> ; filtros opcionais.
PUT	<code>/api/v1/user/EditUser/{id}</code>	<code>{"name":"Ana Souza","email":"novo@example.com"}</code> ; campos opcionais. O <code>{id}</code> deve ser o do usuário autenticado.
PUT	<code>/api/v1/user/EditPassword/{id}</code>	<code>{"new_password":"nova-senha"}</code> . O <code>{id}</code> deve ser o do usuário autenticado.
DELETE	<code>/api/v1/user/DeleteUser/{id}</code>	ID do usuário na URL. Só é possível excluir a própria conta.

Social · prefixo `/api/v1/social`

MÉTODO	ROTA	PARÂMETRO / BODY
POST	<code>/api/v1/social/StartFollowing</code>	<code>{"followee_id":"ID_DE_OUTRO_USUARIO"}</code>
POST	<code>/api/v1/social/Unfollow</code>	<code>{"followee_id":"ID_DO_USUARIO"}</code>
GET	<code>/api/v1/social/ListFollowers/{id}</code>	ID do usuário alvo na URL.
GET	<code>/api/v1/social/ListFollowing/{id}</code>	ID do usuário alvo na URL.

O usuário que segue é extraído do token; não envie `follower_id`. As rotas de listagem recebem explicitamente o ID do usuário cujos seguidores/seguimentos serão consultados.

Autorização por propriedade: `EditUser`, `EditPassword` e `DeleteUser` só atuam sobre a conta do próprio usuário autenticado; um `{id}` diferente do token é rejeitado. Detalhes na seção 07.

Nas rotas acima, substitua cada marcador entre chaves, como `{id}`, por um ID real de teste.

Hábitos

Base do Gateway: `http://localhost:8084`. Os IDs de hábito e rotina são retornados nas respostas de criação. Datas são enviadas em formato RFC 3339, por exemplo `2026-10-09T15:00:00Z`.

Hábitos · prefixo `/api/v1/habit` — apenas hábitos do usuário autenticado

MÉTODO	ROTA	PARÂMETRO / BODY
POST	<code>/api/v1/habit/</code>	<code>{"name": "Caminhada", "description": "30 min", "image_url": "https://..."};</code> name necessário.
GET	<code>/api/v1/habit/</code>	Lista hábitos do usuário autenticado.
GET	<code>/api/v1/habit/{id}</code>	ID do hábito. Só retorna hábito do usuário autenticado.
GET	<code>/api/v1/habit/routine/{id}</code>	ID da rotina. A rotina e os hábitos retornados devem ser do usuário autenticado.
PUT	<code>/api/v1/habit/{id}</code>	<code>{"name": "Caminhada atualizada"};</code> edição parcial. Só o próprio hábito.
DELETE	<code>/api/v1/habit/{id}</code>	ID do hábito. Só o próprio hábito.
POST	<code>/api/v1/habit/{id}/complete</code>	<code>{"completed_at": "2026-10-09T15:00:00Z"}</code>
DELETE	<code>/api/v1/habit/{id}/complete</code>	Mesmo body <code>completed_at</code> para desfazer.
GET	<code>/api/v1/habit/{id}/logs</code>	<code>?start_date=2026-10-01T00:00:00Z&end_date=2026-10-10T00:00:00Z</code> ID e datas obrigatórios (RFC 3339).

Leitura compartilhada · prefixo `/api/v1/habit/shared`

MÉTODO	ROTA	COMPORTAMENTO
GET	<code>/api/v1/habit/shared/{id}</code>	Busca um hábito pelo ID, mesmo que pertença a outra pessoa. NOVA
GET	<code>/api/v1/habit/shared/routine/{id}</code>	Lista os hábitos associados à rotina. Retorna somente os hábitos cujo dono é o mesmo da rotina. NOVA

Leitura de outros usuários: `GET /habit/{id}` e `GET /habit/routine/{id}` retornam apenas recursos do usuário autenticado. Para ler o hábito ou a rotina de outra pessoa, use as rotas `/shared`, que são somente leitura e exigem autenticação.

Datas de conclusão e filtros de histórico usam formato RFC 3339.

Rotinas

Base do Gateway: `http://localhost:8084`. As rotinas pertencem ao usuário autenticado. É possível associar hábitos existentes e registrar conclusões com data.

Rotinas · prefixo `/api/v1/routine` — apenas rotinas do usuário autenticado

MÉTODO	ROTA	PARÂMETRO / BODY
POST	<code>/api/v1/routine/</code>	<code>{"name": "Rotina matinal"}</code>
GET	<code>/api/v1/routine/</code>	Lista rotinas do usuário autenticado.
GET	<code>/api/v1/routine/{id}</code>	ID da rotina. Só retorna rotina do usuário autenticado.
PUT	<code>/api/v1/routine/{id}</code>	<code>{"name": "Nova rotina"}</code> . Só a própria rotina.
DELETE	<code>/api/v1/routine/{id}</code>	ID da rotina. Só a própria rotina.
POST	<code>/api/v1/routine/{id}/habits</code>	<code>{"habit_id": "ID_DO_HABITO"}</code> . A rotina e o hábito devem ser do usuário autenticado.
DELETE	<code>/api/v1/routine/{id}/habits/{habitID}</code>	IDs da rotina e do hábito no caminho. Ambos devem ser do usuário autenticado.
POST	<code>/api/v1/routine/{id}/complete</code>	<code>{"completed_at": "2026-10-09T15:00:00Z"}</code>
DELETE	<code>/api/v1/routine/{id}/complete</code>	Mesmo body <code>completed_at</code> para desfazer.
GET	<code>/api/v1/routine/{id}/logs</code>	?start_date=2026-10-01T00:00:00Z&end_date=2026-10-10T00:00:00Z ID e datas obrigatórios (RFC 3339).

Leitura compartilhada · prefixo `/api/v1/routine/shared`

MÉTODO	ROTA	COMPORTAMENTO
GET	<code>/api/v1/routine/shared/{id}</code>	Busca uma rotina pelo ID, mesmo que pertença a outra pessoa. NOVA

Leitura de outros usuários: GET `/routine/{id}` retorna apenas rotinas do usuário autenticado. Para consultar a rotina de outra pessoa, use `/shared/{id}` (somente leitura, exige autenticação).

Os endpoints de desfazer conclusão usam DELETE com body JSON. Confirme que o cliente HTTP preserve o body em requisições DELETE.

Habit e Routine são domínios distintos na API e compartilham o serviço gRPC interno.

Estatísticas, erros e integração

Base do Gateway: `http://localhost:8084`. As estatísticas são vinculadas ao usuário autenticado. Não é necessário enviar o ID do usuário na URL ou no body.

Estatísticas · prefixo `/api/v1/stats`

MÉTODO	ROTA	PARÂMETRO / BODY
POST	<code>/api/v1/stats</code>	Cria as estatísticas do usuário; sem body.
GET	<code>/api/v1/stats</code>	Consulta estatísticas; sem body.
DELETE	<code>/api/v1/stats</code>	Remove estatísticas; sem body.
POST	<code>/api/v1/stats/habits/{id}/completions</code>	<code>{"completed_at": "2026-10-09T15:00:00Z"}</code> . O hábito deve ser do usuário do token.
DELETE	<code>/api/v1/stats/habits/{id}/completions</code>	Mesmo body <code>completed_at</code> para desfazer. Hábito do usuário do token.
POST	<code>/api/v1/stats/routines/{id}/completions</code>	<code>{"completed_at": "2026-10-09T15:00:00Z"}</code> . A rotina deve ser do usuário do token.
DELETE	<code>/api/v1/stats/routines/{id}/completions</code>	Mesmo body <code>completed_at</code> para desfazer. Rotina do usuário do token.

O **stats-service** confirma que o hábito ou a rotina pertence ao usuário do token antes de registrar ou desfazer a conclusão.

Erros comuns

STATUS	SIGNIFICADO	AÇÃO SUGERIDA
400	Body JSON inválido, obrigatório ausente ou argumento inválido.	Confira formato, campos e datas.
401	Token ausente ou inválido no header <code>Authorization</code> .	Faça login e envie o access token usando o esquema Bearer.
404	Recurso não encontrado (dependendo do serviço).	Confira o ID e se o recurso existe.
413	Body acima do limite da rota.	Reduza o tamanho do JSON.
500	Erro interno, dependência indisponível ou configuração incompleta.	Confira logs e configuração dos serviços.

Boas práticas para testar

1. Use uma conta e registros descartáveis de desenvolvimento.
2. Guarde IDs retornados por signup, criação de hábitos e criação de rotinas.
3. Configure variáveis de ambiente no Postman para base URL e token; não exporte tokens reais.
4. Para acessar via Nginx, troque o host do Gateway por `http://localhost`; a porta 8084 é o acesso direto ao Gateway.

Segurança: tokens JWT validados no Gateway, refresh tokens no Redis, gRPC com TLS entre os serviços e autorização por propriedade dos recursos (seção 07). Este guia descreve a API observada no código.

Autorização e propriedade de recursos

O usuário autenticado só consulta e altera os próprios recursos. O ID do usuário vem sempre do token, nunca do cliente.

Resumo por domínio

DOMÍNIO	ROTAS	REGRA APLICADA
Usuário	PUT EditUser/{id}, PUT EditPassword/{id}, DELETE DeleteUser/{id}	O {id} da URL deve ser o do usuário autenticado.
Hábito	GET/PUT/DELETE /habit/{id}, POST/DELETE /habit/{id}/complete, GET /habit/{id}/logs, GET /habit/routine/{id}	Somente hábitos do usuário autenticado. Em routine/{id}, a rotina e os hábitos retornados devem ser do usuário.
Rotina	GET/PUT/DELETE /routine/{id}, POST/DELETE /routine/{id}/complete, GET /routine/{id}/logs	Somente rotinas do usuário autenticado.
Rotina ↔ hábito	POST /routine/{id}/habits, DELETE /routine/{id}/habits/{habitID}	A rotina e o hábito devem ser do usuário autenticado.
Estatísticas	POST/DELETE /stats/habits/{id}/completions, POST/DELETE /stats/routines/{id}/completions	O stats-service confirma que o hábito/rotina pertence ao usuário do token.
Leitura compartilhada	GET /habit/shared/{id}, GET /habit/shared/routine/{id}, GET /routine/shared/{id}	Somente leitura; permite ver recursos de outros donos.

Rotas novas · /shared

MÉTODO	ROTA	COMPORTAMENTO
GET	/api/v1/routine/shared/{id}	Busca uma rotina pelo ID, mesmo que pertença a outra pessoa. NOVA
GET	/api/v1/habit/shared/{id}	Busca um hábito pelo ID, mesmo que pertença a outra pessoa. NOVA
GET	/api/v1/habit/shared/routine/{id}	Lista os hábitos associados à rotina. Retorna somente os hábitos cujo dono é o mesmo da rotina. NOVA

Para ler hábitos e rotinas de outros usuários, use as rotas /shared. As demais rotas de hábito e rotina retornam apenas os recursos do usuário autenticado.

Como testar

1. Crie dois usuários, A e B, e um hábito e uma rotina para o usuário B.
2. Com o token de A, tente PUT, DELETE e complete no hábito e na rotina de B. Todas as tentativas são recusadas.
3. Com o token de A, tente EditUser, EditPassword e DeleteUser usando o ID de B. Todas são recusadas.
4. Com o token de A, chame GET /habit/shared/{id} no hábito de B. Retorna o hábito.
5. Com o token de A, chame GET /habit/{id} no hábito de B. É recusado.

Contrato dos serviços

O Gateway consome estes serviços gRPC com TLS. São interfaces internas da stack; não são endpoints HTTP nem rotas públicas para o app.

SERVIÇO	ENDEREÇO DENTRO DO DOCKER	PROTO PACKAGE
UserService	user-service:8080	user
SocialService	social-service:8081	social
HabitService	habit-service:8082	habit
RoutineService	habit-service:8082	habit
StatsService	stats-service:8083	stats

Transporte: gRPC sobre TLS, usando certificados do projeto. As portas 8080–8083 são internas à rede Docker; o Compose de produção não as publica no host. A porta 8084 publicada pelo Compose atende HTTP do Gateway, não gRPC.

Convenções dos contratos

1. Tipos `string`, `bool`, números inteiros e listas são campos Protobuf comuns.
2. `google.protobuf.Timestamp` representa uma data/hora absoluta em UTC; use RFC 3339 ao converter para JSON.
3. Campos marcados como `optional` distinguem ausência de valor de `string` vazia.
4. O Gateway injeta o usuário autenticado em chamadas de domínio quando o método REST deriva o ID do token. O cliente gRPC interno recebe os campos declarados no contrato.

Para testar diretamente com um cliente gRPC fora do Docker, será necessário publicar/encaminhar a porta apropriada e configurar a confiança no certificado CA. Isso não está habilitado pelo Compose de produção padrão.

Contratos definidos pelos arquivos Protocol Buffers em `backend/services/shared/proto`.

UserService

Package user · endereço interno user-service:8080 · transporte TLS.

Modelo User

User { id: string, name: string, email: string }

MÉTODO RPC	REQUEST	RESPONSE
GetUserByID	GetUserByIDRequest { user_id: string }	GetUserByIDResponse { user: User }
GetUsersByIDs	GetUsersByIDsRequest { user_ids: string[] }	GetUsersByIDsResponse { users: User[] }
SearchUser	SearchUserRequest { name?: string, email?: string }	SearchUserResponse { user: User[] }
EditUser	EditUserRequest { user_id: string, name?: string, email?: string }	EditUserResponse { user: User }
EditPassword	EditPasswordRequest { user_id: string, new_password: string }	EditPasswordResponse { success: bool }
DeleteUser	DeleteUserRequest { user_id: string }	DeleteUserResponse { success: bool }

Mapeamento REST: GetUserByID, GetUsersByIDs, SearchUser, EditUser, EditPassword e DeleteUser são expostos pelas rotas HTTP da seção Usuários. As rotas de alteração só aceitam o ID do usuário autenticado.

O endpoint REST de busca em lote recebe o campo ids; o Gateway o converte para user_ids no request Protobuf.

Contrato: services/shared/proto/user/user.proto · porta interna 8080

SocialService

Package social · endereço interno social-service:8081 · transporte TLS.

Modelo User

User { id: string, name: string, email: string }

MÉTODO RPC	REQUEST	RESPONSE
StartFollowing	StartFollowingRequest { follower_id: string, followee_id: string }	StartFollowingResponse { success: bool }
Unfollow	UnfollowRequest { follower_id: string, followee_id: string }	UnfollowResponse { success: bool }
ListFollowers	ListFollowersRequest { user_id: string }	ListFollowersResponse { users: User[] }
ListFollowing	ListFollowingRequest { user_id: string }	ListFollowingResponse { users: User[] }

Mapeamento REST: os quatro métodos são expostos pelas rotas Social da seção Gateway. Nas operações de seguir/deixar de seguir, o Gateway preenche `follower_id` com o ID do usuário autenticado e recebe do cliente apenas `followee_id`.

O serviço rejeita IDs inválidos, usuários inexistentes e tentativas de seguir a si mesmo. Erros gRPC do Social são atualmente apresentados pelo Gateway como HTTP 500.

Contrato: services/shared/proto/social/social.proto · porta interna 8081

HabitService

Package habit · endereço interno habit-service:8082 · transporte TLS.

Modelos

Habit { id, user_id, name, description, image_url: string; created_at: Timestamp }

HabitLog { habit_id: string; completed_at: Timestamp }

MÉTODO RPC	REQUEST	RESPONSE
CreateHabit	{ habit: Habit }	{ habit: Habit }
GetHabitByID	{ habit_id: string }	{ habit: Habit }
ListHabitsByUser	{ user_id: string }	{ habits: Habit[] }
ListHabitsByRoutine	{ routine_id: string }	{ habits: Habit[] }
EditHabit	{ habit_id: string, name?: string, description?: string, image_url?: string }	{ habit: Habit }
DeleteHabit	{ habit_id: string }	{ success: bool }
MarkHabitCompleted	{ habit_id: string, completed_at: Timestamp }	{ success: bool }
UnmarkHabitCompleted	{ habit_id: string, completed_at: Timestamp }	{ success: bool }
GetHabitLogs	{ habit_id: string, start_date: Timestamp, end_date: Timestamp }	{ logs: HabitLog[] }

O Gateway define user_id na criação e na listagem por usuário a partir do token autenticado. As demais operações usam o ID do hábito ou da rotina indicado na rota HTTP.

Contrato: services/shared/proto/habit/v4/habit.proto · porta interna 8082

RoutineService

Package habit · compartilha o endereço interno habit-service:8082 com HabitService · TLS.

Modelos

Routine { id, user_id, name: string; created_at: Timestamp }

RoutineLog { routine_id: string; completed_at: Timestamp }

MÉTODO RPC	REQUEST	RESPONSE
CreateRoutine	{ routine: Routine }	{ routine: Routine }
GetRoutineByID	{ routine_id: string }	{ routine: Routine }
ListRoutinesByUser	{ user_id: string }	{ routines: Routine[] }
EditRoutine	{ routine_id: string, name?: string }	{ routine: Routine }
DeleteRoutine	{ routine_id: string }	{ success: bool }
AddHabitToRoutine	{ routine_id: string, habit_id: string }	{ success: bool }
RemoveHabitFromRoutine	{ routine_id: string, habit_id: string }	{ success: bool }
MarkRoutineCompleted	{ routine_id: string, completed_at: Timestamp }	{ success: bool }
UnmarkRoutineCompleted	{ routine_id: string, completed_at: Timestamp }	{ success: bool }
GetRoutineLogs	{ routine_id: string, start_date: Timestamp, end_date: Timestamp }	{ logs: RoutineLog[] }

user_id para criação/listagem vem da identidade autenticada no Gateway. Operações de vínculo e conclusão recebem os IDs correspondentes e timestamps do cliente REST.

Contrato: services/shared/proto/habit/v4/habit.proto · porta interna 8082

StatsService

Package stats · endereço interno stats-service:8083 · transporte TLS.

Modelo UserStats

```
UserStats { user_id: string; completed_habits, completed_routines, current_habit_streak,  
longest_habit_streak, current_routine_streak, longest_routine_streak: int32; created_at, updated_at:  
Timestamp }
```

MÉTODO RPC	REQUEST	RESPONSE
CreateUserStats	{ user_id: string }	{ stats: UserStats }
GetUserStats	{ user_id: string }	{ stats: UserStats }
DeleteUserStats	{ user_id: string }	{ success: bool }
RegisterHabitCompletion	{ user_id: string, habit_id: string, completed_at: Timestamp }	{ success: bool }
UndoHabitCompletion	{ user_id: string, habit_id: string, completed_at: Timestamp }	{ success: bool }
RegisterRoutineCompletion	{ user_id: string, routine_id: string, completed_at: Timestamp }	{ success: bool }
UndoRoutineCompletion	{ user_id: string, routine_id: string, completed_at: Timestamp }	{ success: bool }

Nas chamadas do Gateway, `user_id` vem do token; o cliente fornece o ID do hábito/rotina e a data de conclusão. O contrato gRPC inclui os três campos no request de registro e remoção. O serviço confirma que o hábito ou a rotina pertence ao `user_id` antes de registrar ou desfazer a conclusão.

Chamadas disponíveis

Criação, consulta e remoção das estatísticas do usuário; registro e reversão de conclusões de hábitos e rotinas. Os endpoints HTTP correspondentes estão na seção Estatísticas.

O serviço Stats tem definições Proto versionadas. Esta referência descreve o contrato atual gerado e consumido pela aplicação; confira o arquivo Proto e código gerado em caso de divergência.

Contrato: services/shared/proto/stats · porta interna 8083